

Inter-Service Communication in Spring Boot

When one backend service needs to communicate with another backend service, common approaches include:

```
REST / HTTP
├─ RestTemplate
├─ OpenFeign
└─ WebClient

Message-based
├─ Kafka
├─ RabbitMQ
└─ other brokers
```

We'll focus first on the HTTP clients.

1. HTTP REST Communication

Suppose we have:

```
Order Service
    ↓
Payment Service
```

The Order Service needs to call:

```
POST /payments
```

with:

```
{
  "orderId": "ORD101",
  "amount": 5000
}
```

The Payment Service returns:

```
{
  "transactionId": "TX1001",
  "status": "SUCCESS"
}
```

This is **REST-based inter-service communication**.

REST is the communication style/protocol approach; `RestTemplate` and `Feign` are tools clients can use to make those HTTP calls.

2. RestTemplate

`RestTemplate` is a Spring HTTP client for **synchronous, blocking** HTTP calls.

Example

```
PaymentResponse response =  
    restTemplate.postForObject(  
        "http://payment-service/payments",  
        request,  
        PaymentResponse.class  
    );
```

The flow is:

```
Order Service  
  ↓  
RestTemplate  
  ↓  
HTTP POST  
  ↓  
Payment Service  
  ↓  
HTTP Response  
  ↓  
Order Service
```

The calling thread waits for the response.

exchange()

For more control, you commonly see:

```
HttpHeaders headers = new HttpHeaders();  
headers.setContentType(MediaType.APPLICATION_JSON);  
headers.setBearerAuth(token);  
  
HttpEntity<PaymentRequest> entity =  
    new HttpEntity<>(request, headers);  
  
ResponseEntity<PaymentResponse> response =  
    restTemplate.exchange(  
        paymentUrl,  
        HttpMethod.POST,  
        entity,  
        PaymentResponse.class  
    );
```

`exchange()` allows you to specify:

- HTTP method
- headers
- request body
- response type

3. Advantages of RestTemplate

Simple

Mature
Easy to understand
Fine for synchronous applications

But the code can become repetitive.

For example:

```
restTemplate.exchange(...)  
restTemplate.exchange(...)  
restTemplate.exchange(...)
```

You have to manually deal with:

- URLs
- HTTP methods
- headers
- serialization
- response handling
- exceptions

This is where **Feign Client** becomes attractive.

4. OpenFeign / Feign Client

Feign lets you define an HTTP client as a **Java interface**.

Instead of writing HTTP client code yourself:

```
@FeignClient(name = "payment-service")  
public interface PaymentClient {  
  
    @PostMapping("/payments")  
    PaymentResponse createPayment(  
        @RequestBody PaymentRequest request);  
}
```

Then simply:

```
PaymentResponse response =  
    paymentClient.createPayment(request);
```

Spring/Feign generates the implementation that performs the HTTP call.

Conceptually:

```
Your Code  
↓  
PaymentClient interface  
↓  
Feign  
↓  
HTTP  
↓  
Payment Service
```

5. Why Feign Is Easier

Compare the two.

RestTemplate

```
ResponseEntity<PaymentResponse> response =  
    restTemplate.exchange(  
        url,  
        HttpMethod.POST,  
        entity,  
        PaymentResponse.class  
    );
```

Feign

```
PaymentResponse response =  
    paymentClient.createPayment(request);
```

Feign is **declarative**.

You describe:

┆ "This method represents this HTTP endpoint."

Feign handles the HTTP client implementation.

6. Feign Interface Example

```
@FeignClient(name = "payment-service")  
public interface PaymentClient {  
  
    @GetMapping("/payments/{id}")  
    PaymentResponse getPayment(  
        @PathVariable("id") String id);  
  
    @PostMapping("/payments")  
    PaymentResponse createPayment(  
        @RequestBody PaymentRequest request);  
}
```

Usage:

```
@Service  
public class OrderService {  
  
    private final PaymentClient paymentClient;  
  
    public OrderService(PaymentClient paymentClient) {  
        this.paymentClient = paymentClient;  
    }  
  
    public PaymentResponse pay(PaymentRequest request) {  
        return paymentClient.createPayment(request);  
    }  
}
```

Notice that there is no explicit:

```
HttpClient
URL connection
JSON conversion
```

logic in the service.

7. Feign + Service Discovery

Feign becomes especially useful in microservices when combined with service discovery/load balancing.

Instead of:

```
http://10.10.1.20:8080
```

you can use a logical service name:

```
@FeignClient(name = "payment-service")
```

Conceptually:

```
Order Service
  ↓
payment-service
  ↓
Service Discovery
  ↓
┌───┴───┐
│   ↓   │
└─ Instance 1 Instance 2 Instance 3
```

The exact mechanism depends on your Spring Cloud/discovery setup.

8. Feign + Eureka

Suppose:

```
Eureka
├─ payment-service instance 1
├─ payment-service instance 2
└─ payment-service instance 3
```

Feign can work with the Spring Cloud load-balancing/discovery infrastructure to resolve:

```
payment-service
```

to an available instance.

So you don't hard-code IP addresses.

9. RestTemplate vs Feign

This is highly likely to be asked.

	RestTemplate	Feign
Style	Imperative	Declarative
Code	More boilerplate	Less boilerplate
Interface-based	No	Yes
Service discovery integration	Possible	Commonly integrated
HTTP configuration	More manual	More declarative/configurable
Readability	Lower for many clients	Usually cleaner

Important:

Feign is still making an HTTP call. It doesn't change REST into a different communication mechanism.

10. WebClient

You should know this because interviewers may ask:

“What would you use in a modern Spring application?”

`WebClient` is Spring's HTTP client designed for **non-blocking/reactive** communication.

Example:

```
Mono<PaymentResponse> response =
    webClient.post()
        .uri("/payments")
        .bodyValue(request)
        .retrieve()
        .bodyToMono(PaymentResponse.class);
```

Conceptually:

```
RestTemplate
→ synchronous/blocking

Feign
→ declarative HTTP client

WebClient
→ supports non-blocking/reactive HTTP
```

Don't say:

“WebClient is always better.”

The right choice depends on the application's architecture and workload.

11. When Would You Choose Each?

RestTemplate

Use when:

```
Existing application
Synchronous communication
Simple HTTP integration
```

Feign

Use when:

```
Many service-to-service APIs
Spring Cloud environment
Want declarative clients
Want less HTTP boilerplate
```

WebClient

Use when:

```
Reactive/non-blocking architecture
High concurrency with I/O
Need reactive pipelines
```

12. Synchronous Nature

Both `RestTemplate` and a typical Feign invocation are **synchronous/blocking from the caller's perspective**.

Example:

```
BPS
↓
Feign / RestTemplate
↓
Hydra
↓
wait
↓
response
```

This is different from asynchronous messaging:

```
BPS
↓
Kafka
↓
Consumer
```

13. Handling HTTP Errors

Suppose Payment Service returns:

```
500 Internal Server Error
```

Your client needs to deal with that appropriately.

With `RestTemplate`, exceptions may be raised for HTTP error responses depending on the configured error handler.

With Feign, you can configure/customize error decoding.

Conceptually:

```
HTTP 4xx/5xx
  ↓
Client
  ↓
Exception
  ↓
Application error handling
```

Don't simply catch everything and convert it to "payment failed."

A 500 response, timeout, connection failure, and an explicit business failure can mean different things.

14. Timeout

A serious backend application should configure timeouts.

There are generally at least two important categories:

```
Connection timeout
→ How long to establish connection?

Read/response timeout
→ How long to wait for response?
```

Example:

```
BPS
  ↓
Hydra
  ↓
no response
  ↓
timeout
```

This is particularly important for your payment system.

15. Retry

Suppose:

```
BPS → Hydra
      ↓
    timeout
```

Should you retry?

Not automatically.

For:

```
GET /payments/123
```

retry may often be relatively safe.

For:

```
POST /payments
```

blind retry can create duplicate processing.

This is why payment systems require:

```
Idempotency
+
safe retry policy
+
transaction state
+
reconciliation
```

16. Circuit Breaker

Suppose Hydra is continuously failing:

```
BPS → Hydra ✕
BPS → Hydra ✕
BPS → Hydra ✕
...
```

Without protection, BPS continues consuming threads/connections waiting for Hydra.

A circuit breaker can move:

```
CLOSED
  ↓
Repeated failures
  ↓
OPEN
```

and temporarily stop calls.

Eventually:

```
OPEN
  ↓
```

```
HALF-OPEN
↓
test call
↓
healthy → CLOSED
```

Common Spring ecosystem choice:

```
Resilience4j
```

17. REST Communication vs Message Queue

Now compare HTTP with messaging.

REST

```
BPS
↓
HTTP request
↓
Hydra
↓
HTTP response
```

Caller generally waits.

Messaging

```
BPS
↓
Message Broker
↓
Consumer
```

Producer doesn't necessarily wait for consumer processing to finish.

18. When Would Messaging Be Better?

Imagine:

```
Payment completed
```

and you need to:

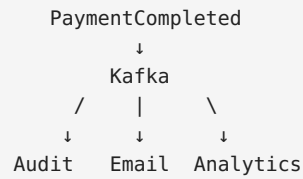
```
send notification
update analytics
create audit event
```

These may not need to block the payment response.

You could publish:

```
PaymentCompleted
```

and let multiple consumers process it.



This gives looser runtime coupling.

19. Message Delivery Problems

Messaging introduces its own challenges.

Duplicate messages

```
PaymentCompleted
PaymentCompleted
```

Consumer must be idempotent.

Consumer failure

```
Message
↓
Consumer
↓
DB unavailable
```

Need retry/DLQ handling.

Ordering

```
INITIATED
SUCCESS
REVERSED
```

should not become:

```
SUCCESS
INITIATED
REVERSED
```

for a workflow where ordering matters.

20. REST vs Messaging — Interview Table

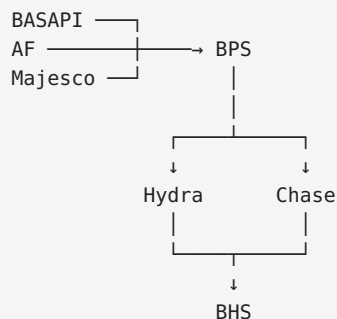
	REST	Messaging
Communication	HTTP	Broker
Model	Request/response	Publish/consume

Typical behavior	Synchronous	Asynchronous
Coupling	Higher runtime dependency	Lower runtime dependency
Response	Immediate	Potentially later
Buffering	Limited	Strong
Retry	Client-side	Consumer/broker workflow
Complexity	Lower	Higher
Good for	Immediate result	Async workflows/events

21. Now Apply This to BPS

This is the part that should be **specific to your project**.

Your architecture is:



BPS is the **payment service and orchestration/translation bridge** between the source systems, Majesco billing system, and payment infrastructure.

BPS:

```

Validate
↓
Orchestrate
↓
Translate
↓
Select vendor
↓
Call payment infrastructure
↓
Post to Majesco
↓
Reconcile
↓
History through BHS
  
```

22. BPS → Hydra/Chase

You told me BPS uses **RestTemplate**.

So your actual flow is:

```
BPS
↓
Read token
↓
vendor field
↓
Factory
↓
Strategy
├── Hydra Strategy
│   ↓
│   RestTemplate
│   ↓
│   Hydra
└── Chase Strategy
    ↓
    RestTemplate
    ↓
    Chase
```

This is an excellent example to explain in an interview.

23. Why Strategy + Factory Here?

Suppose the token says:

```
{
  "vendor": "HYDRA"
}
```

BPS can select:

```
HYDRA
↓
Hydra Strategy
```

For:

```
{
  "vendor": "CHASE"
}
```

it selects:

```
CHASE
↓
Chase Strategy
```

Conceptually:

```
PaymentStrategy strategy =
    paymentStrategyFactory
        .getStrategy(token.getVendor());

PaymentResponse response =
    strategy.process(request);
```

Then each strategy can use the appropriate `RestTemplate` integration.

This gives you a very strong connection between:

```
Factory
+
Strategy
+
Polymorphism
+
REST
+
RestTemplate
```

24. Why Not Feign in BPS?

Since your actual BPS implementation uses `RestTemplate`, **don't say you use Feign**.

An interviewer may ask:

“Why didn't you use Feign?”

Unless you know the actual project decision, don't invent an answer like:

“Feign was not performant.”

A safer answer is:

“Our existing BPS implementation uses `RestTemplate` for the vendor integrations. I'm familiar with Feign as a declarative alternative for service-to-service HTTP communication, but I wouldn't claim it is part of our current implementation.”

Then you can discuss how you'd evaluate Feign:

```
RestTemplate
→ existing synchronous integration

Feign
→ declarative service client
→ less HTTP boilerplate
→ potentially useful if the architecture standardizes on it
```

25. BPS and Recurring Payments

Your recurring payment flow is separate from online payments:

```
Recurring Payment Batch
↓
Sequential chunks
↓
BPS
↓
Vendor Strategy
↓
Hydra / Chase
```

Do **not** describe this as a message-queue system unless your implementation actually uses a queue.

The important distinction is:

```
Online payment
→ synchronous request/response flow

Recurring payment
→ batch processing
→ sequential chunks
```

26. BPS Failure Scenario

Suppose:

```
BPS
↓
Hydra
↓
timeout
```

Your interviewer may ask:

“What happens?”

You should discuss:

1. RestTemplate timeout
2. Exception handling
3. Current transaction state
4. Whether retry is safe
5. Vendor transaction/reference status
6. Reconciliation
7. Majesco/BHS consistency

And the key payment-specific point:

“A timeout does not prove that Hydra did not process the payment.”

That is why reconciliation is important.

27. BPS vs Message Queue

If interviewer asks:

“Could BPS use Kafka instead of RestTemplate to call Hydra?”

Answer carefully:

“Technically an asynchronous architecture could be designed, but it would change the payment-processing model. Our current vendor integration is synchronous because BPS needs the vendor response as part of its orchestration. Moving that interaction to messaging would introduce asynchronous state management, correlation, retries, duplicate-message handling and potentially eventual consistency. I would choose it only if those trade-offs were

| justified by the business and operational requirements.”

That’s a strong 3-year answer.

28. Questions You Should Prepare

General

1. What is inter-service communication?
2. REST vs message queue?
3. Synchronous vs asynchronous?
4. What is RestTemplate?
5. What is Feign Client?
6. What is WebClient?
7. RestTemplate vs Feign?
8. Feign vs WebClient?
9. Why use service discovery?
10. How does a timeout occur?

Resilience

11. How do you handle timeout?
12. When should you retry?
13. Why is retry dangerous for payment POST requests?
14. What is idempotency?
15. What is a circuit breaker?
16. What happens if downstream service is unavailable?

Messaging

17. What is Kafka?
18. Queue vs topic?
19. What is consumer group?
20. What is offset?
21. What is at-least-once delivery?
22. What is DLQ?
23. How do you handle duplicate messages?

BPS-specific

24. How does BPS communicate with Hydra?
 25. How does BPS communicate with Chase?
 26. Why does BPS use RestTemplate ?
 27. How does BPS decide which vendor to call?
 28. Explain Factory + Strategy in your BPS implementation.
 29. What happens if Hydra times out?
 30. What happens if Hydra succeeds but BPS doesn’t receive the response?
 31. What happens if BHS is unavailable?
 32. How does reconciliation help?
 33. How is recurring-payment processing different?
 34. Why are recurring payments processed in sequential chunks?
-

29. Interview-Ready Answer for Your BPS

“In BPS, inter-service communication is primarily REST-based for the synchronous payment-provider integrations. BPS uses `RestTemplate` to communicate with external payment providers such as Hydra and Chase. The payment token contains a `vendor` field, which BPS uses with Factory and Strategy patterns to select the appropriate vendor implementation. That strategy builds the vendor-specific request and invokes the external provider through `RestTemplate`. BPS then translates and processes the provider response as part of its orchestration, including the required posting to Majesco, reconciliation and interaction with BHS for history.

I’m also familiar with alternatives such as OpenFeign, which provides a declarative interface-based approach to HTTP communication, and WebClient, which supports reactive/non-blocking communication. However, our BPS implementation uses `RestTemplate`, so I would describe Feign and WebClient as alternative approaches rather than claim they are part of our current implementation. For asynchronous workloads, message brokers such as Kafka can be used, but our recurring payment processing currently uses sequential chunk-based batch processing rather than a message-queue model.”

That is the level of explanation I’ll use for the remaining topics: **teach the general concept first, then explicitly connect it to what you actually do in BPS**, without mixing theoretical technologies into your project history.